

```
!pip install Levenshtein
```

```
Collecting Levenshtein
  Downloading levenshtein-0.27.4-cp313-cp313-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (3.7 kB)
Collecting rapidfuzz<4.0.0,>=3.9.0 (from Levenshtein)
  Downloading rapidfuzz-3.14.6-cp313-cp313-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (12 kB)
Download levenshtein-0.27.4-cp313-cp313-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (158 kB)
----- 158.7/158.7 kB 1.4 MB/s eta 0:00:00
Download rapidfuzz-3.14.6-cp313-cp313-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (3.2 MB)
----- 3.2/3.2 MB 16.4 MB/s eta 0:00:00
Installing collected packages: rapidfuzz, Levenshtein
Successfully installed Levenshtein-0.27.4 rapidfuzz-3.14.6
```

Double-click (or enter) to edit

```
import json
import math
```

Double-click (or enter) to edit

```
with open('text_segmentation_dataset.json', 'r') as f:
    data = json.load(f)
```

```
counts = data['word_counts']
tot_words = data['metadata']['total_corpus_words']
cases = data['test_cases']
```

```
vocab = set(counts.keys())
log_probs = {w: math.log(c / tot_words) for w, c in counts.items()}
```

```
def lev_dist(s1, s2):
    dp = {}

    def solve(i, j):
        if (i, j) in dp:
            return dp[(i, j)]

        if i == len(s1):
            return len(s2)-j
        if j == len(s2):
            return len(s1)-i

        if s1[i] == s2[j]:
            res = solve(i+1, j+1)
        else:
            res = 1 + min(
                solve(i+1, j),
                solve(i, j+1),
                solve(i+1, j+1)
            )

        dp[(i, j)] = res
        return res

    return solve(0, 0)
```

```
def seg_greedy(txt, vocab):
    n = len(txt)
    res = []
    i=0

    while i < n:
        match = ""
        for j in range(i+1, n+1):
            sub = txt[i:j]
            if sub in vocab and len(sub) > len(match):
                match = sub

        if match:
            res.append(match)
            i += len(match)
        else:
            res.append(txt[i])
            i += 1
```

```

return " ".join(res)

def seg_dp(txt, log_probs):
    n = len(txt)
    dp = [-float('inf')]*(n+1)
    dp[0] = 0.0
    bp = [-1] * (n+1)

    for i in range(1, n + 1):
        for j in range(i):
            w = txt[j:i]
            if w in log_probs:
                score = dp[j] + log_probs[w]
                if score > dp[i]:
                    dp[i] = score
                    bp[i] = j

    res = []
    curr=n
    while curr > 0:
        prev = bp[curr]
        if prev == -1:
            return seg_greedy(txt, set(log_probs.keys()))
        res.append(txt[prev:curr])
        curr = prev

    return " ".join(reversed(res))

def evaluate(cases, vocab, log_probs, limit=None, show=False):
    data = cases[:limit] if limit else cases
    tot = len(data)

    g_corr, d_corr = 0, 0
    g_dist, d_dist = 0, 0

    for idx, c in enumerate(data, 1):
        inp, gt = c['input'], c['ground_truth']

        g_out = seg_greedy(inp, vocab)
        d_out = seg_dp(inp, log_probs)

        g_ok = (g_out == gt)
        d_ok = (d_out == gt)

        if g_ok:
            g_corr += 1
        if d_ok:
            d_corr += 1

        ed_g = lev_dist(g_out, gt)
        ed_d = lev_dist(d_out, gt)

        g_dist += ed_g
        d_dist += ed_d

        if show:
            print(f"[{idx}] {inp}")
            print(f"Truth: {gt}")
            print(f"Greedy: {g_out} | Match: {g_ok} | Edit: {ed_g}")
            print(f"DP: {d_out} | Match: {d_ok} | Edit: {ed_d}\n")

    g_acc = (g_corr/tot)
    d_acc = (d_corr/tot)
    g_avg_d = g_dist/tot
    d_avg_d = d_dist/tot

    print(f"Evaluated: {tot} samples")
    print(f"Greedy - Acc: {g_acc:.2f}%, Avg Edit: {g_avg_d:.4f}")
    print(f"DP - Acc: {d_acc:.2f}%, Avg Edit: {d_avg_d:.4f}")

    return {
        "greedy_acc": g_acc,
        "greedy_edit": g_avg_d,
        "dp_acc": d_acc,
        "dp_edit": d_avg_d
    }

def eval10(cases, vocab, log_probs):
    return evaluate(cases, vocab, log_probs, limit=10, show=True)

```

```
def seg_user(txt):
    clean = txt.strip().lower().replace(" ", "")
    print(f"Input: {clean}")
    print(f"Greedy: {seg_greedy(clean, vocab)}")
    print(f"DP: {seg_dp(clean, log_probs)}\n")
```

```
eval10(cases, vocab, log_probs)
evaluate(cases, vocab, log_probs)
```

```
seg_user("thisisapublicschoolinnewyork")
seg_user("therearemanydifferentpeoplestandingaroundthesquare")
```

```
[1] itthatthecitytakestepstothisproblem
Truth: it that the city take steps to this problem
Greedy: it that the city takes t e p s to this problem | Match: False | Edit: 5
DP: it that the city take steps to this problem | Match: True | Edit: 0

[2] oftitlelawwasalsobythe
Truth: of title law was also by the
Greedy: of title law was also by the | Match: True | Edit: 0
DP: of title law was also by the | Match: True | Edit: 0

[3] failuretodotothiswillcontinuetoplaceaon
Truth: failure to do this will continue to place a on
Greedy: failure to do this will continue top l a c e a on | Match: False | Edit: 5
DP: failure to do this will continue to place a on | Match: True | Edit: 0

[4] onotherthethat
Truth: on other the that
Greedy: on other the that | Match: True | Edit: 0
DP: on other the that | Match: True | Edit: 0

[5] williamforfromhiswifeincourt
Truth: william for from his wife in court
Greedy: william for from his wife in court | Match: True | Edit: 0
DP: william for from his wife in court | Match: True | Edit: 0

[6] thecouplewasmarrried
Truth: the couple was married
Greedy: the couple was married | Match: True | Edit: 0
DP: the couple was married | Match: True | Edit: 0

[7] theyhaveasonwilliamandadaughterof
Truth: they have a son william and a daughter of
Greedy: they have as on william and a daughter of | Match: False | Edit: 2
DP: they have as on william and a daughter of | Match: False | Edit: 2

[8] forthesaidthatanpropertyhasbeenagreedupon
Truth: for the said that an property has been agreed upon
Greedy: forth e said that an property has been agreed upon | Match: False | Edit: 2
DP: for the said that an property has been agreed upon | Match: True | Edit: 0

[9] thetheasandhisageas
Truth: the the as and his age as
Greedy: the the as and his age as | Match: True | Edit: 0
DP: the the as and his age as | Match: True | Edit: 0

[10] ithisageasandplaceofas
Truth: it his age as and place of as
Greedy: it his age as and place of as | Match: True | Edit: 0
DP: it his age as and place of as | Match: True | Edit: 0

Evaluated: 10 samples
Greedy - Acc: 0.60%, Avg Edit: 1.4000
DP - Acc: 0.90%, Avg Edit: 0.2000
Evaluated: 1000 samples
Greedy - Acc: 0.69%, Avg Edit: 1.2900
DP - Acc: 0.98%, Avg Edit: 0.0280
Input: thisisapublicschoolinnewyork
Greedy: this is a public school in new york
```

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

```
seg_user("theyoungchildwalkeddownthedarkstreet")
```

```
Input: theyoungchildwalkeddownthedarkstreet
Greedy: they o u n g child walked down the dark street
DP: the young child walked down the dark street
```

Start coding or [generate](#) with AI.